



Why learn with us ?



Training From Experts



One - on – One Guidance



100% Practical



Real-Time Projects & Case Studies



Placement Preparation Support



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Contact :- 8085896740

 @ hello_world_institute

 Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Inheritance

Inheritance is the ability to 'inherit' features or attributes from already written classes into newer classes we make. This promotes code reusability which is considered one of the best industrial coding practices as it makes the codebase modular.

In python inheritance, new class/es inherit from older class/es. The new class/es copy all functions and attributes of the older class into itself without rewriting the syntax in the new class/es. These new classes are also called derived classes, and old classes are called base classes.

For example – The word inheritance is often used in biology to symbolize the transfer of genetic traits from parents to their children. Similarly, we have parent classes (Base classes) and child classes (derived classes). In Inheritance, we derive classes from other already existing classes. The existing classes are the parent/base classes from which the attributes and methods are inherited in the child classes.

Advantages of Inheritance in Python

- **Modular Codebase:** Increases modularity i.e. breaking down codebase into modules which makes it easier to understand. Here, each class that we define becomes a separate module that can be separately inherited by one or many classes.
- **Code Reusability:** the child class copies all the attributes and methods of the parent class into its own class and use. This saves time and coding effort by not rewriting them thus following modularity paradigms.
- **Less Development:** and Maintenance Costs: changes just need to be made in the base class, all derived classes will automatically follow.

Disadvantages of Inheritance in Python

- **Decreases the Execution Speed:** loading multiple classes because they are interdependent on each other
- **Tightly Coupled Classes:** this means that even though parent classes can be executed independently, child classes cannot be executed without defining their parent classes.



OFFLINE CLASSES



ONLINE LECTURE

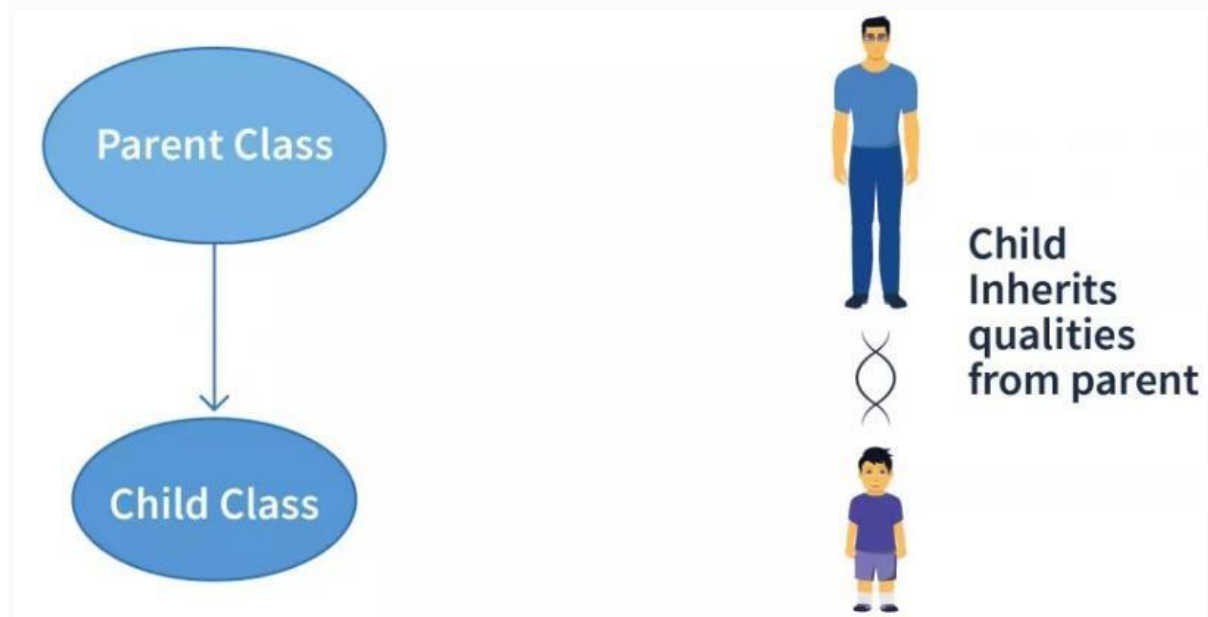


RECORDED ACCESS

Types of Inheritance in Python

Single Inheritance

Single Inheritance is the simplest form of inheritance where a single child class is derived from a single parent class. Due to its candid nature, it is also known as simple inheritance.



Example

```
class parent:          # parent class
    def func1:
        print("Hello Parent")
class child(parent):    # child class
    def func2:          # we include the parent class
        print("Hello Child") # as an argument in the child class
# Driver Code
test = child()          # object created
test.func1()            # parent method called via child object
test.func2()            # child method called
```



OFFLINE CLASSES



ONLINE LECTURE

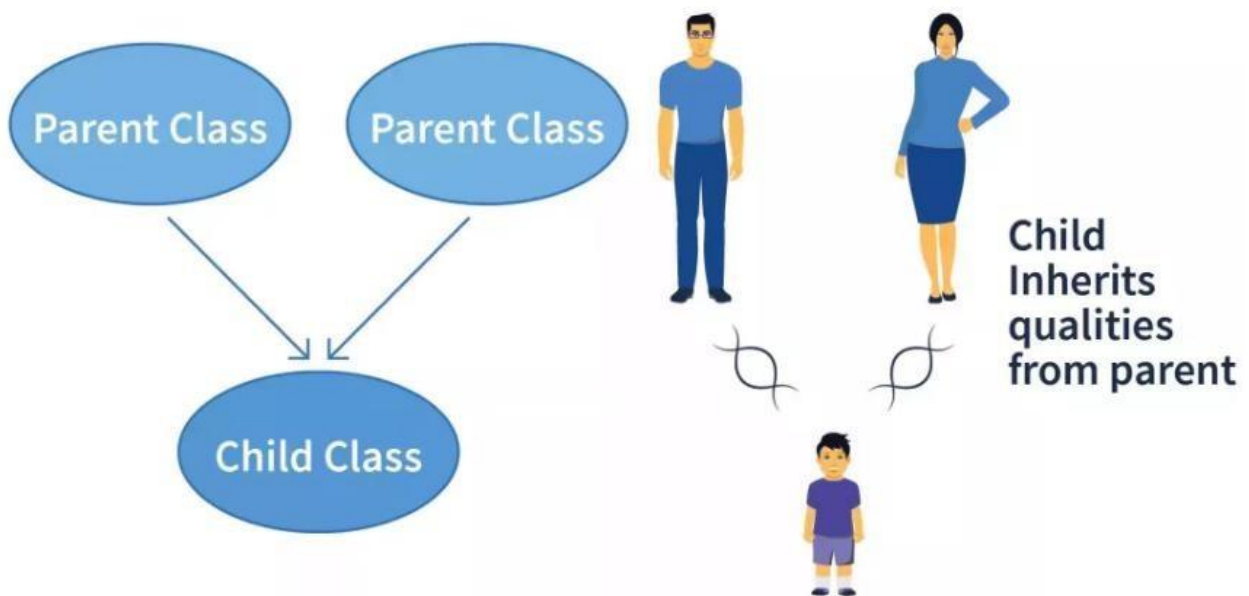


RECORDED ACCESS

Multiple Inheritance

In multiple inheritance, a single child class is inherited from two or more parent classes. This means the child class has access to all the methods and attributes of all the parent classes.

However, if two parents have the same “named” methods, the child class performs the method of the first parent in order of reference. To get a better understanding of which class’s methods shall be executed first, we can use the Method Resolution Order function (mro). This tells the order in which the child class is interpreted to visit the other classes.



Example:

multiple inheritance example

```
class parent1:          # first parent class
    def func1:
        print("Hello Parent1")
class parent2:          # second parent class
    def func2:
        print("Hello Parent2")
class child(parent1, parent2):
    def func3:
        print("Hello Child")
```



OFFLINE CLASSES



ONLINE LECTURE

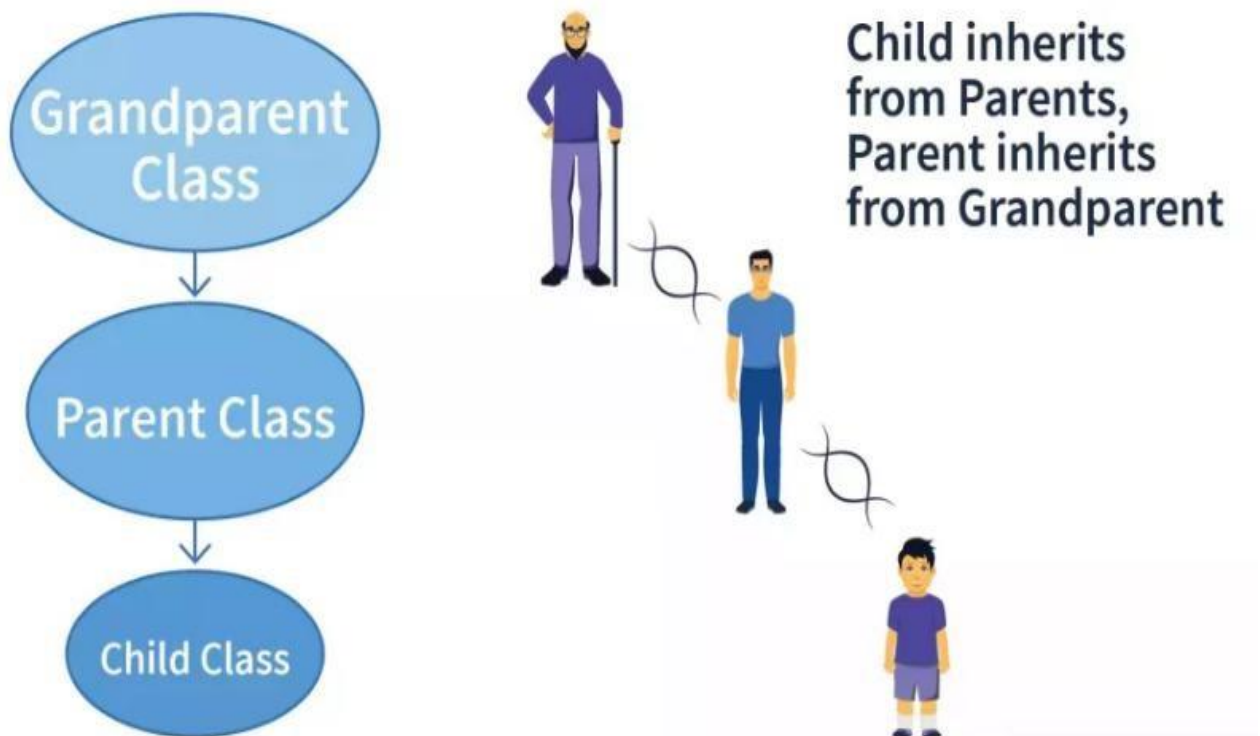


RECORDED ACCESS

```
# Driver Code
test = child()    # object created
test.func1()
test.func2()
test.func3()
# to find the order of classes visited by the child class, we use __mro__ on the
child class
print(child.__mro__)
```

Multi-Level Inheritance

In multilevel inheritance, we go beyond just a parent-child relation. We introduce grandchildren, great-grandchildren, grandparents, etc. Basically, till now we saw only two levels of inheritance with a superior parent class/es and a derived class/es, but here we can have multiple levels where the parent class/es itself is derived from another class/es.



Example:



OFFLINE CLASSES



ONLINE LECTURE

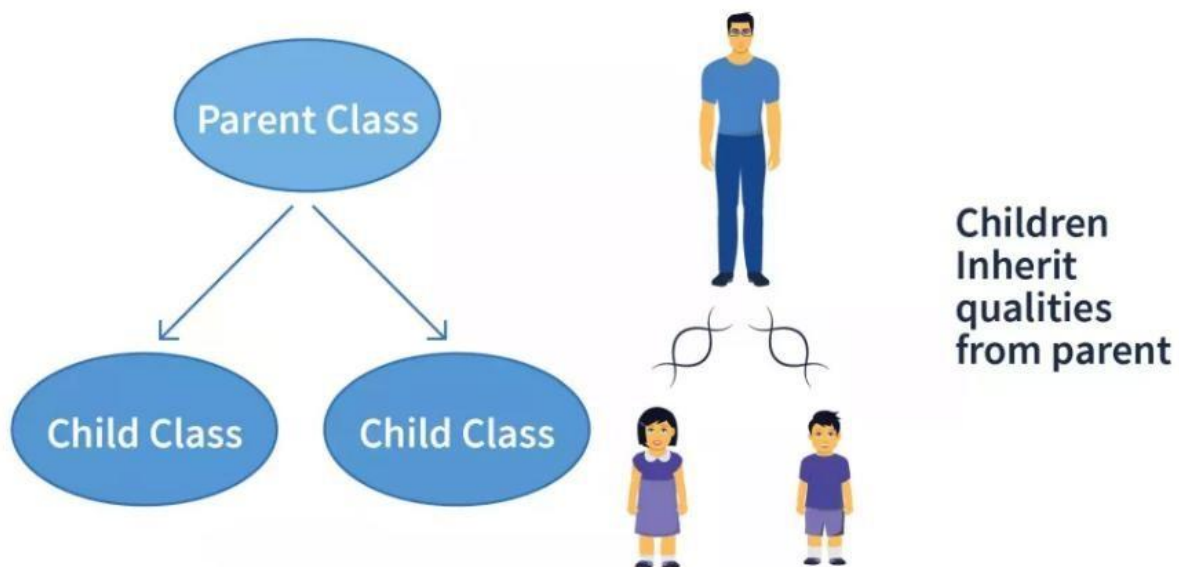


RECORDED ACCESS

```
# multi-level inheritance example
class grandparent:      # first level
    def func1:
        print("Hello Grandparent")
class parent(grandparent):  # second level
    def func2:
        print("Hello Parent")
class child(parent):      # third level
    def func3:
        print("Hello Child")
# Driver Code
test = child()           # object created
test.func1()             # 3rd level calls 1st level
test.func2()             # 3rd level calls 2nd level
test.func3()             # 3rd level calls 3rd level
```

Hierarchical Inheritance

Hierarchical Inheritance is the right opposite of multiple inheritance. This means that there are multiple derived child classes from a single parent class.



Example:

hierarchical inheritance example



OFFLINE CLASSES



ONLINE LECTURE

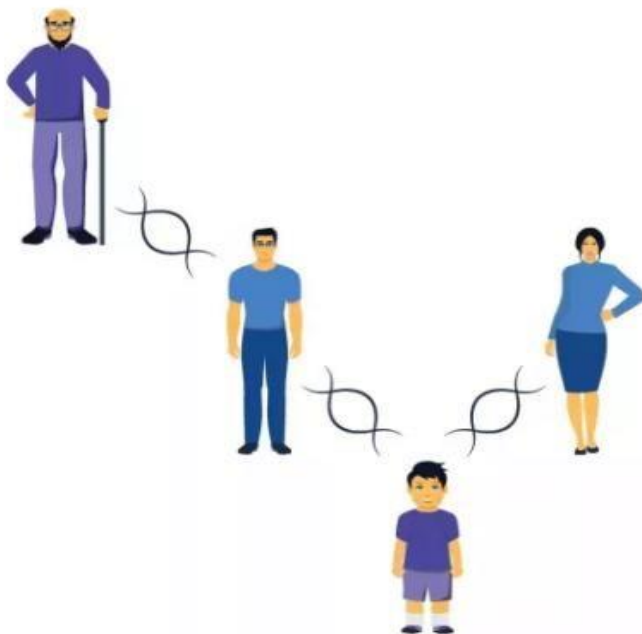


RECORDED ACCESS

```
class parent:                # parent class
    def func1:
        print("Hello Parent")
class child1(parent):        # first child class
    def func2:
        print("Hello Child1")
class child2(parent):        # second child class
    def func3:
        print("Hello Child2")
# Driver Code
test1 = child1()             # objects created
test2 = child2()
test1.func1()                # child1 calling parent method
test1.func2()                # child1 calling its own method
test2.func1()                # child2 calling parent method
test2.func3()                # child2 calling its own method
```

Hybrid Inheritance

Hybrid Inheritance is the mixture of two or more different types of inheritance. Here we can have many to many relations between parent classes and child classes with multiple levels.



**Child inherits
from grandparents
and parents**



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Example:

hybrid inheritance example

```
class parent1:          # first parent class
    def func1:
        print("Hello Parent")
class parent2:          # second parent class
    def func2:
        print("Hello Parent")
class child1(parent1):   # first child class
    def func3:
        print("Hello Child1")
class child2(child1, parent2): # second child class
    def func4:
        print("Hello Child2")
# Driver Code
test1 = child1()         # object created
test2 = child2()
test1.func1()            # child1 calling parent1 method
test1.func2()            # child1 calling its own method
test2.func1()            # child2 calling parent1 method
test2.func2()            # child2 calling parent2 method
test2.func3()            # child2 calling child1 method
test2.func4()            # child2 calling its own method
```

In this example, we show a combination of three types of python inheritance.

Parent1 -> Child1 : Single Inheritance

Parent1 -> Child1 -> Child2 : Multi – Level Inheritance

Parent1 -> Child2 <- Parent2 : Multiple Inheritance

The diagrammatic explanation of this hybrid inheritance is:



OFFLINE CLASSES

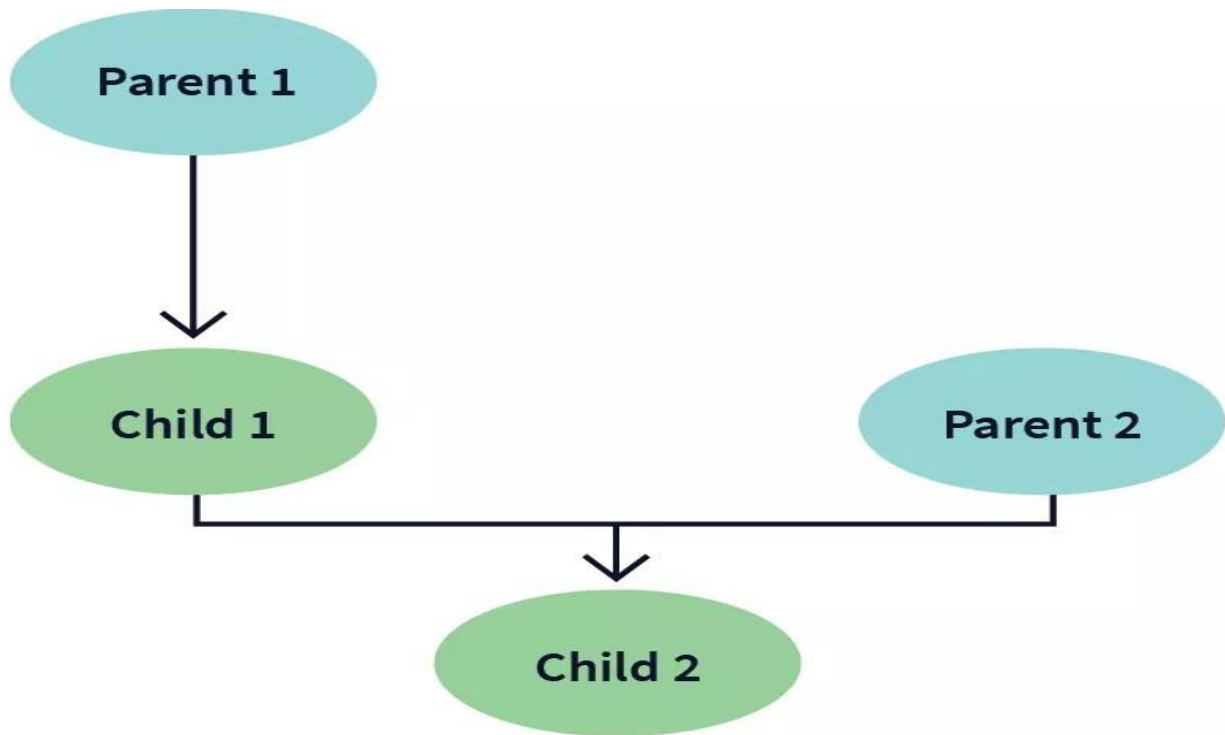


ONLINE LECTURE



RECORDED ACCESS





Special Functions in Python Inheritance

Python is a very versatile and user-friendly language. It provides some amazing inbuilt functions which makes our lives easier when it comes to understanding inheritance especially of a complex nature.

super() function

Method overriding is an ability of any object-oriented programming language that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. This discrepancy is caused due to similar naming convention of the methods. Commonly we can see this situation when the parent's **init()** is overridden by the child's **init()** and hence the child class is unable to inherit attributes from the parent class.

Similarly, we can face this same problem with methods other than **init** but having the same naming convention across parent and child classes.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

One way to solve this, is to call the parent method inside the child method.

solution to method overriding - 1

```
class parent:
    def __init__(self):
        attr1 = 50
        attr2 = 66
class child(parent):
    def __init__(self):
        parent.__init__(self)
        attr3 = 45

test = child()          # object initiated
print (test.attr1)      # parent attribute called via child
```

Another way to solve this problem without explicitly typing out the parent name is to use super(). It automatically references the parent/base class the child class is derived from. This is extremely helpful to call overridden methods in classes with a huge number of methods.

Example:

solution to method overriding - 2

```
class parent:          # parent class
    def display(self):  # display() of parent
        print("Hello Parent")
class child(parent):   # child class
    def display(self):  # display() of child
        super().display() # referencing parent via super()
        print("Hello Child")
test = child()         # object initiated
test.display()         # display of both activated
```



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

